



TEAM PROJECT PRESENTATION

7전8기의 암시장

김소현

고은지

박영수

성기찬

이지민



01

프로젝트 소개

복합 결제와 구독 시스템을 구현한 결제 프로젝트

02

팀 협업 & 개발 전략

효율적인 협업을 위한 역할 분담과 개발 규칙 수립

03

일정 계획

단계별 기능 구현을 위한 개발 일정 관리

04

시스템 설계

결제 정합성을 보장하기 위한 구조 설계

05

핵심 기능

결제 흐름과 복합 결제, 구독 로직의 핵심 구현

06

구현 결과 & 시연

구현된 기능의 실제 동작 과정 확인

07

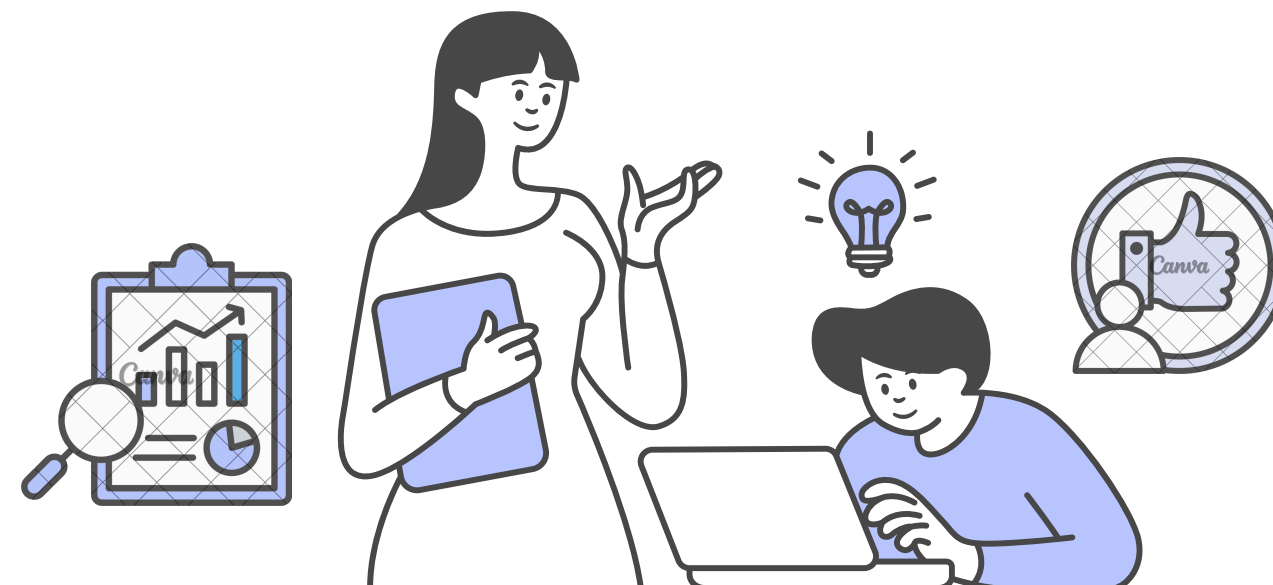
트러블슈팅 & 회고

문제 해결 경험과 프로젝트를 통해 얻은 인사이트

08

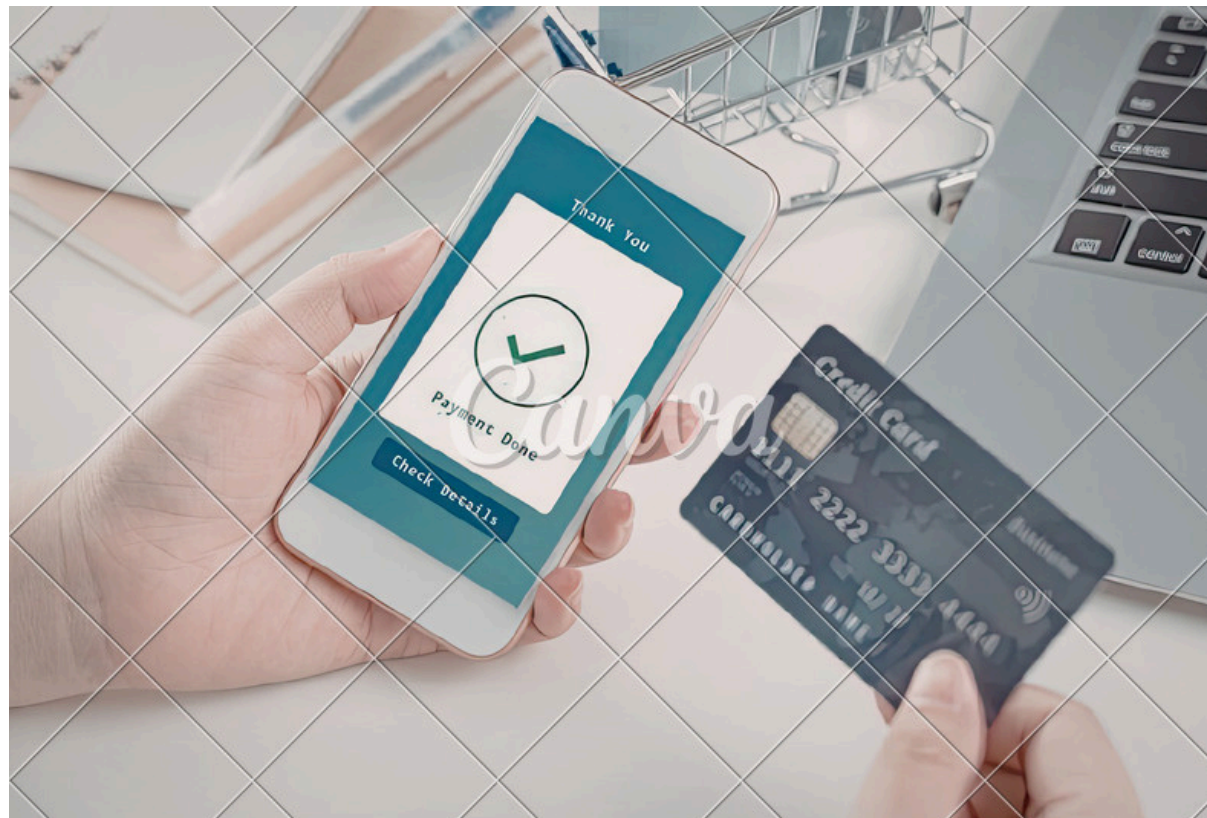
Q&A

궁금한 점이나 구현 및 설계 관련 질문



커머스 복합 결제 및 구독 시스템

단건 결제를 넘어 포인트·멤버십·구독까지 확장 가능한 결제 시스템 구현

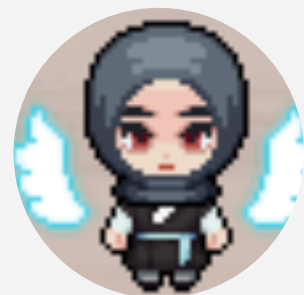


- 01 단건 결제 중심 구조로 다양한 결제 처리에 한계
- 02 포인트/멤버십 기능이 분리되어 사용자 경험 단절
- 03 구독 기반 수익 구조 확장 어려움

02 팀 협업 & 개발 전략



• 팀원 소개

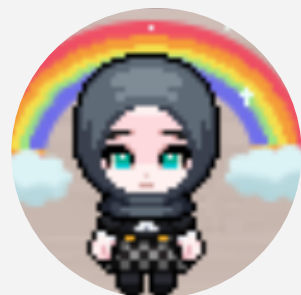


리더/결제
김소현

- 결제 생성 및 결제 확정 처리
- 결제 검증 및 상태 관리
- 환불 요청 및 환불 이력 관리
- 웹훅 처리



일정 관리 및 프로젝트 총괄

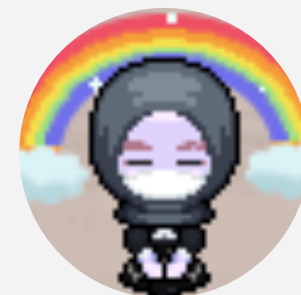


포인트/멤버십
고은지

- 포인트 적립/차감/조회
- 포인트 거래 내역 관리
- 멤버십 등급 조회 및 갱신



회의록 정리 및 발표자료 제작



구독
박영수

- 빌링키 발급 및 관리
- 정기 결제 처리
- 구독 결제 이력 관리

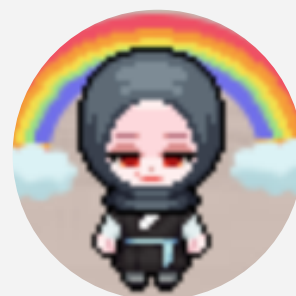


구독 도메인 협업 및 개발 지원

02 팀 협업 & 개발 전략



• 팀원 소개

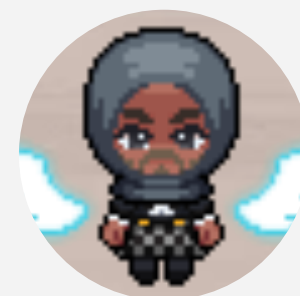


인증/데브옵스
성기찬

- 회원가입/로그인/로그아웃
- 사용자 정보 조회
- AWS 인프라 구축
- CI/CD 구성



개발 방향 조율



상품/주문/구독
이지민

- 상품 목록 및 단건 조회
- 주문 생성 및 주문 내역 조회
- 구독 생성/조회/해지
- 플랜 변경



발표자료 제작 및 회의록 정리

- 팀 컨벤션



프로젝트 Conventions

- ▶ 1 Git 컨벤션
- ▶ 2 프로젝트 패키지 구조
- ▶ 3 네이밍 컨벤션
- ▶ 4 Entity 컨벤션
- ▶ 5 DTO 컨벤션
- ▶ 6 API Response
- ▶ 7 예외 처리 컨벤션
- ▶ 8 Validation 컨벤션
- ▶ 9 로그 컨벤션
- ▶ 10 AWS 배포 구조

▼ 5 DTO 컨벤션

- Request DTO

```
@Builder
public record CreateOrderRequest(
    Long customerId,
    Long productId,
    Integer quantity
) {}
```

- Response DTO

```
@Builder
public record CreateOrderResponse (
    String orderNumber,
    Customer customer,
    Product product,
    String message
){
    public static CreateOrderResponse of(...) {
        return CreateOrderResponse.builder()
            .orderNumber(orderNumber)
            .customer(customer)
            .product(product)
            .message(orderNumber + "의 주문이 접수되었습니다.")
    }
}
```

02 팀 협업 & 개발 전략



• 회의록

월	화	수	목	금
16	17	18	19	
📄 [정기] 12:...	📄 [정기] 10...	📄 10:00	📄 [튜터님] ...	📄 [튜터님]
📄 12:30	📄 12:50	📄 [튜터님] ...	[정기] 10:00	
📄 [튜터님] 1...	📄 14:00			
📄 17:00	📄 16:00			
📄 [튜터님] ...	📄 [튜터님] 1...			

ERD 피드백

- 컬럼 수정
 1. `expiry_at` → `expired_at`
 2. `Date` → `Datetime`
 3. `payments` 에서 `refunded_at` 제거
- `Category` 위로 올리기 (나중에 키로 관리될 수 있음)
- ERD 배치: 주문 관련 테이블은 Order 하위로 배치

API 피드백

- 회원가입: `register` → `signup`
- 등급 정책 조회: URL에서 `policies` 제거
- 포인트 거래내역: `me` 뒤로 이동

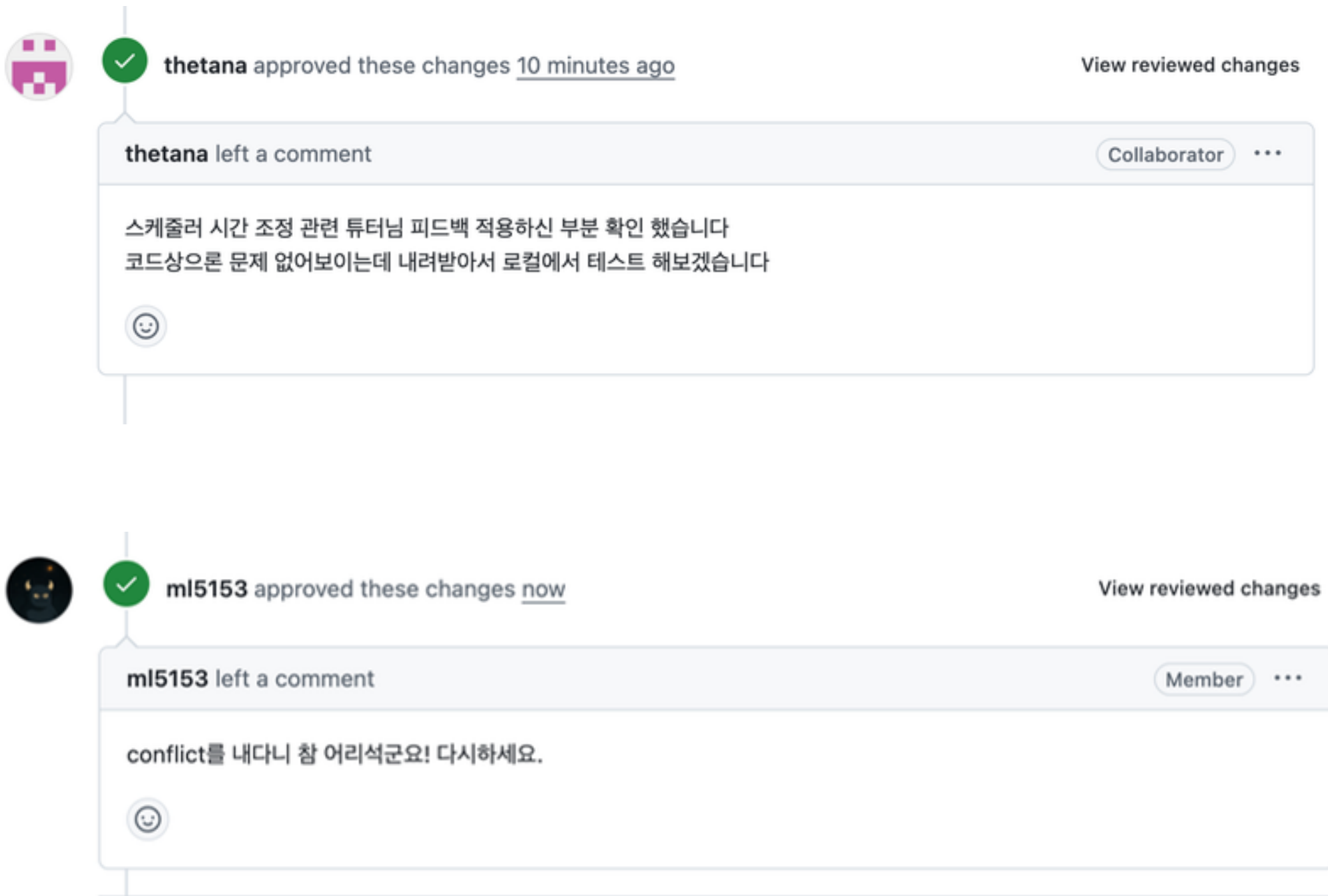
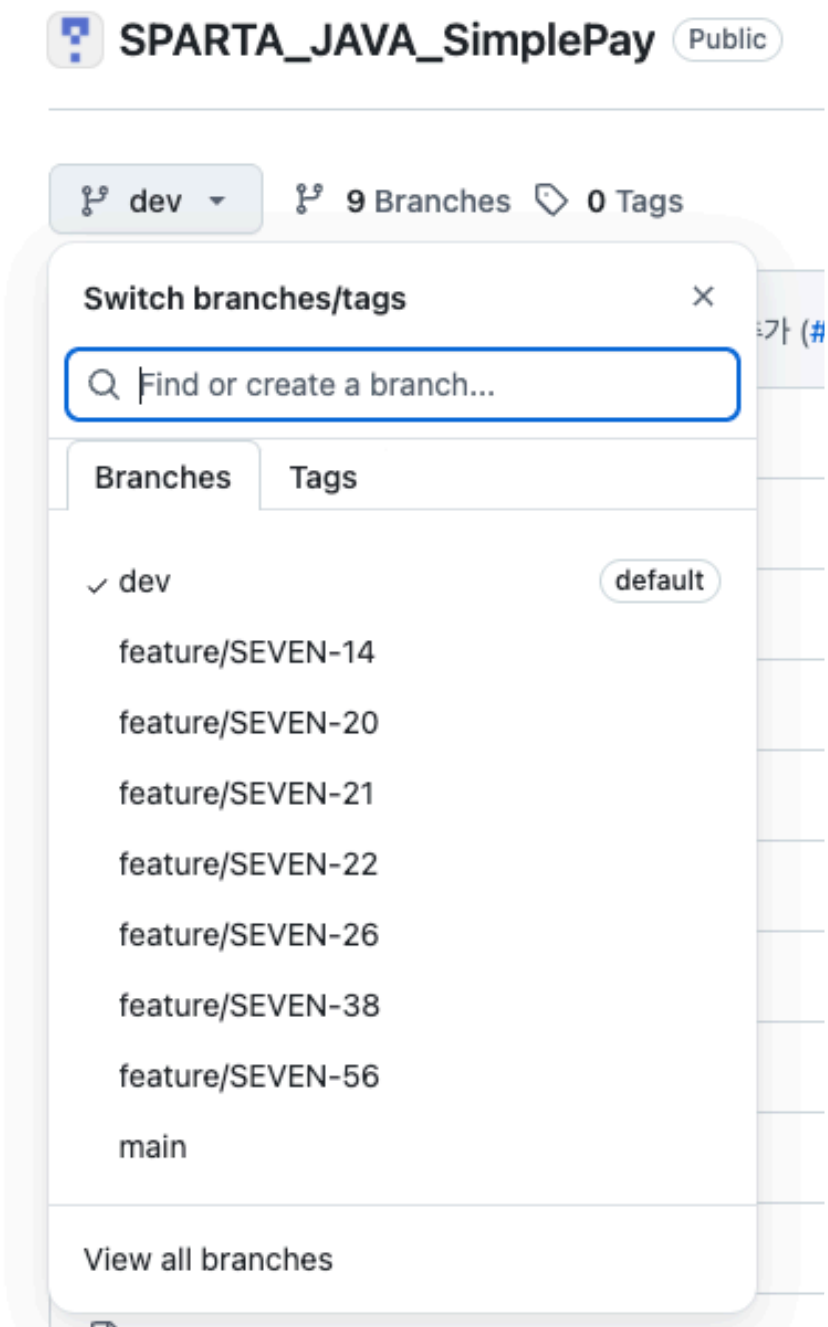
역할 분담 변경

이름	파트
박영수	결제수단 + PortOne 빌링키 + 자동결제 스케줄러 + 청구 이력
이지민	구독 관리 도메인 + 조회/변경 API

02 팀 협업 & 개발 전략



• Git



02 팀 협업 & 개발 전략



• Git

☐	 Cicd #25 by thetana was merged last week • Approved
☐	 fix: 구독 플랜 목록 조회 url 수정 #24 by jiiimni was merged last week • Approved
☐	 feat: 구독 도메인 및 플랜 조회 생성 조회 해지 API #23 by jiiimni was merged last week • Approved
☐	 feat : 결제 취소 및 환불 로직 수정 및 리팩토링 #22 by sohyun0502 was merged last week • Approved
☐	 feat : 결제 확정시 비관적 락 적용 및 커스텀 예외처리 추가 #21 by sohyun0502 was merged last week • Approved
☐	 CI를 먼저 돌려보자! ❌ #20 by thetana was merged last week • Approved
☐	 feat: 테스트 데이터 일부 수정 CICD 셋팅 #19 by thetana was merged last week • Approved
☐	 feat: 포인트 만료 스케줄러 추가 #18 by despis221-cmd was merged last week • Approved
☐	 feat : 결제 검증 조회 기능 수정 및 결제 취소 로직 추가 #17 by sohyun0502 was merged last week • Approved
☐	 feat: 멤버십 등급 정책 조회 및 내 등급 조회 API 추가 #16 by despis221-cmd was merged last week • Approved
☐	 feat: oauth2 적용 #15 by thetana was merged last week • Approved
☐	 feat: 포인트 거래 로직 및 조회 API 추가 #14 by despis221-cmd was merged last week • Approved



jiiimni commented last week • edited ▾

Member ...

작업 내용

- ERD 기준으로 상품(Product), 주문(Order), 주문상품(OrderItem) 엔티티 추가
- 상품(ProductRepository), 주문(OrderRepository) 레포지토리 추가
- 주문/상품 상태 관리를 위한 enum(ProductStatus, OrderStatus)을 추가
- BaseEntity를 상속하도록 반영
- JPA Auditing 사용을 위해 @EnableJpaAuditing 을 적용

주요 변경 사항

Product

- 상품명, 가격, 재고, 설명, 상태, 카테고리 필드 추가

Order

- 사용자(Member) 연관관계 추가
- 주문번호, 총 금액, 상태, 사용 포인트 필드 추가

OrderItem

- 주문/상품 연관관계 추가
- 주문 시점 상품명, 상품가격 스냅샷 필드 추가
- 수량 필드 추가

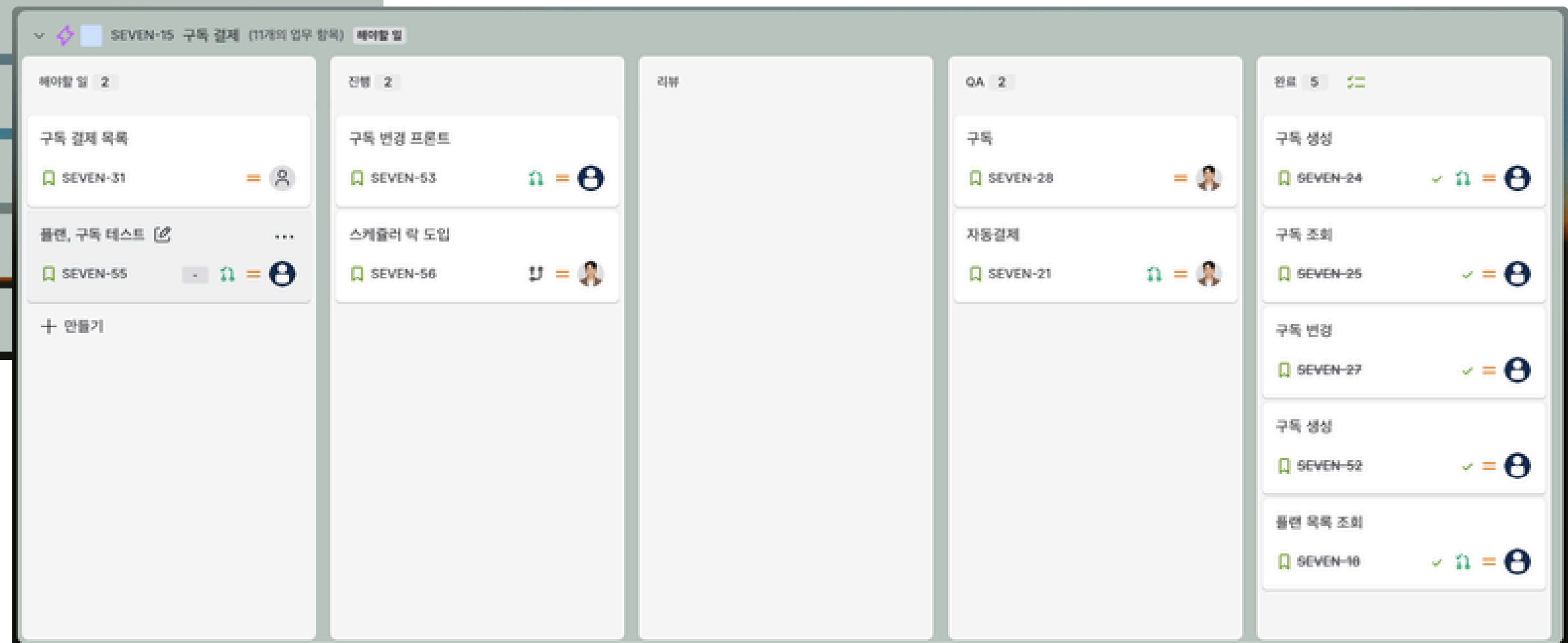
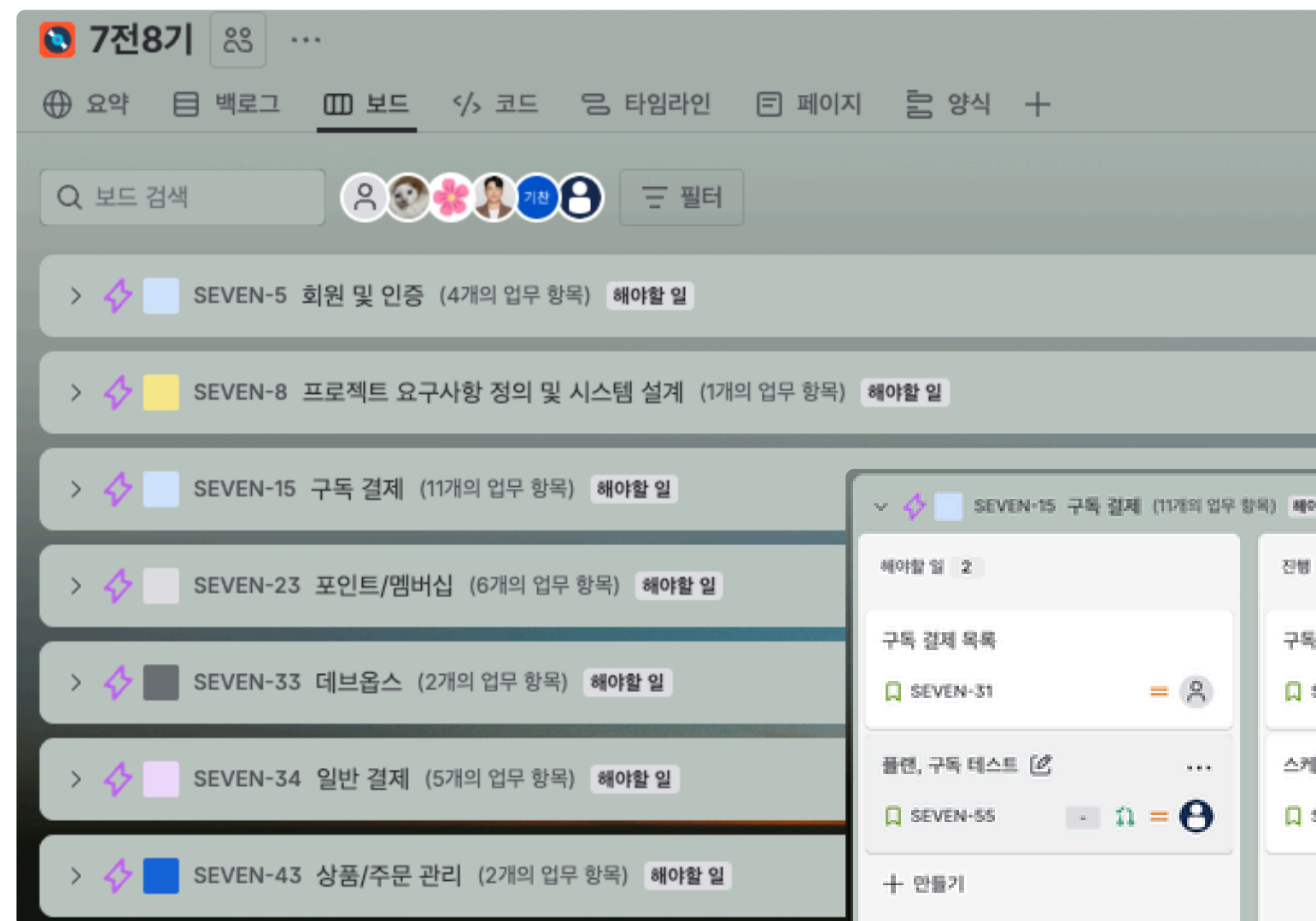
왜 이렇게 구현했는지

- 과제 ERD 기준으로 상품/주문/주문상품 구조를 먼저 맞추는 것이 우선이라고 판단
- 주문 시점의 상품 정보를 위해 productName , productPrice 를 OrderItem에 스냅샷으로 저장하도록 설계

02 팀 협업 & 개발 전략



• Jira



[illegible]

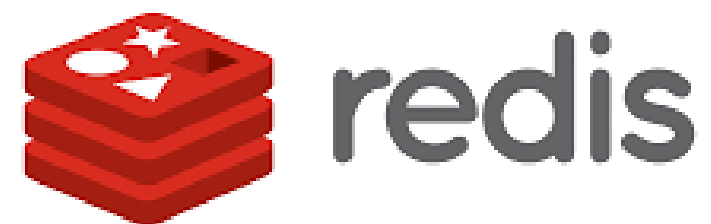
04 시스템 설계 - 기술 스택



Backend



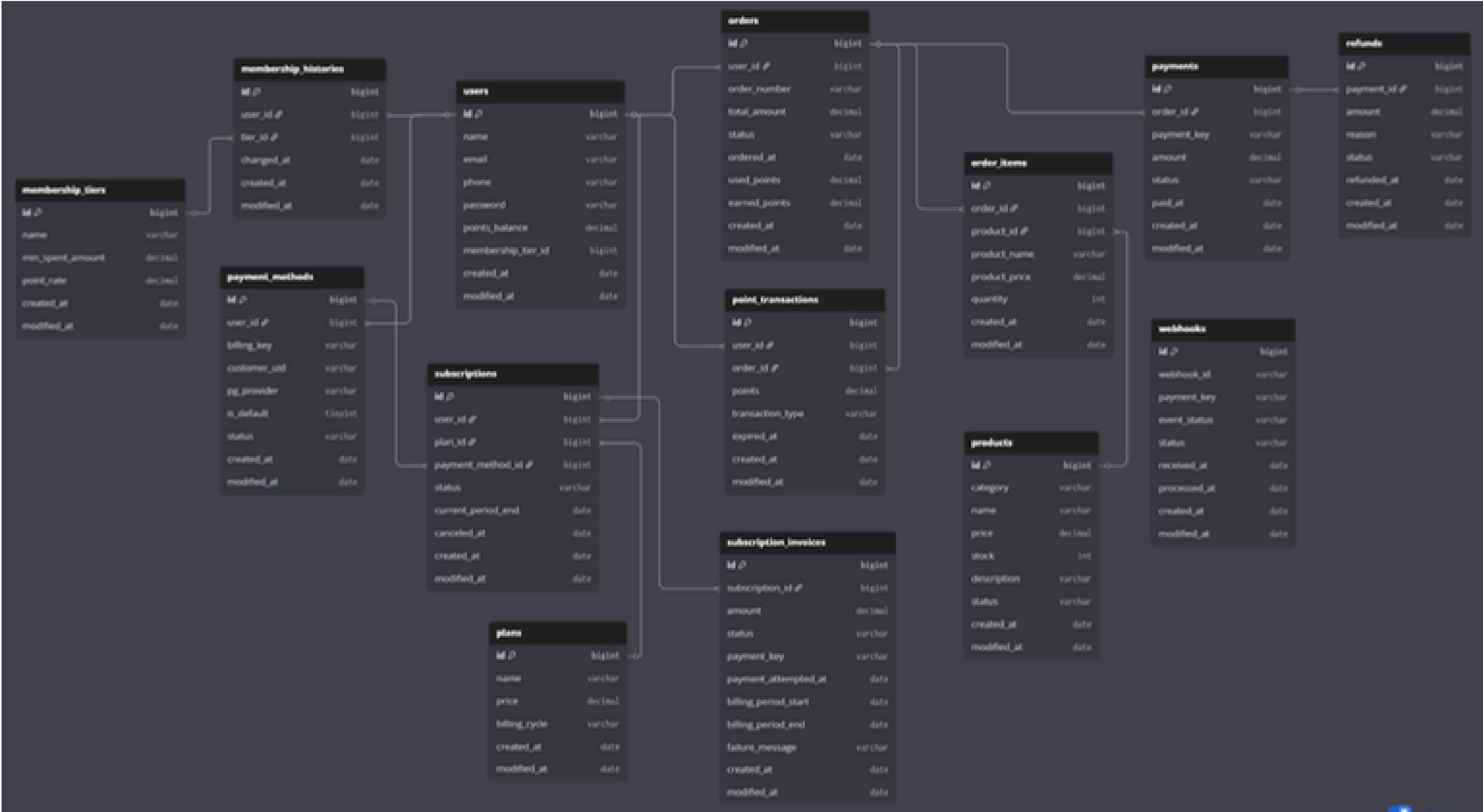
Database



Infra / Tools



04 시스템 설계 - ERD



04 시스템 설계 - API



🔄 진행여부	📍 도메인	📄 기능명	≡ API Path	📌 method	👤 담당자	📁 종류
● 완료	인증	🔒 회원가입	/api/auth/signup	POST	성기찬	클라 → 서버
● 완료	인증	🔒 로그인	/api/auth/login	POST	성기찬	클라 → 서버
● 완료	인증	🔒 로그아웃	/api/auth/logout	POST	성기찬	클라 → 서버
● 완료	인증	👤 내 정보	/api/auth/me	GET	성기찬	클라 → 서버
● 완료	상품	📦 상품 목록 조회	/api/products	GET	이지민	클라 → 서버
● 완료	상품	📦 상품 단건 조회	/api/products/{productId}	GET	이지민	클라 → 서버
● 완료	주문	🛒 주문 생성	/api/orders	POST	이지민	클라 → 서버
● 완료	주문	🛒 주문 목록 조회	/api/orders	GET	이지민	클라 → 서버
● 완료	주문	🛒 주문 단건 조회	/api/orders/{orderId}	GET	이지민	클라 → 서버

기본 커머스 흐름을 기준으로 사용자와 주문 도메인을 분리하여 설계

04 시스템 설계 - API



🔄 진행여부	🌐 도메인	📌 기능명	📄 API Path	📄 method	👤 담당자	📄 종류
● 완료	결제	📄 결제 시도 생성	/api/payments	POST	김소현	클라 → 서버
● 완료	결제	📄 결제창 호출	PortOne.requestPayment()		김소현	클라 → 포트원
● 완료	결제	📄 결제 확정	/api/payments/{paymentId}/confirm	POST	김소현	클라 → 서버
● 완료	결제	📄 결제 검증 조회	api.portone.io/payments/{paymentId}	GET	김소현	서버 → 포트원
● 완료	결제	📄 웹훅	/api/webhook/portone	POST	김소현	포트원 → 서버
● 완료	결제	📄 결제 환불	/api/payments/{paymentId}/cancel	POST	김소현	클라 → 서버
● 완료	결제	📄 결제 취소	api.portone.io/payments/{paymentId}/cancel	POST	김소현	서버 → 포트원
● 완료	포인트/멤버십	📄 내 등급 조회	/api/memberships/me	GET	고은지	클라 → 서버
● 완료	포인트/멤버십	📄 등급 정책 조회	/api/memberships	GET	고은지	클라 → 서버
● 완료	포인트/멤버십	📄 포인트 거래내역	/api/points/transactions/me	GET	고은지	클라 → 서버

결제와 포인트, 멤버십을 연결하여 복합 결제 흐름을 처리하도록 설계

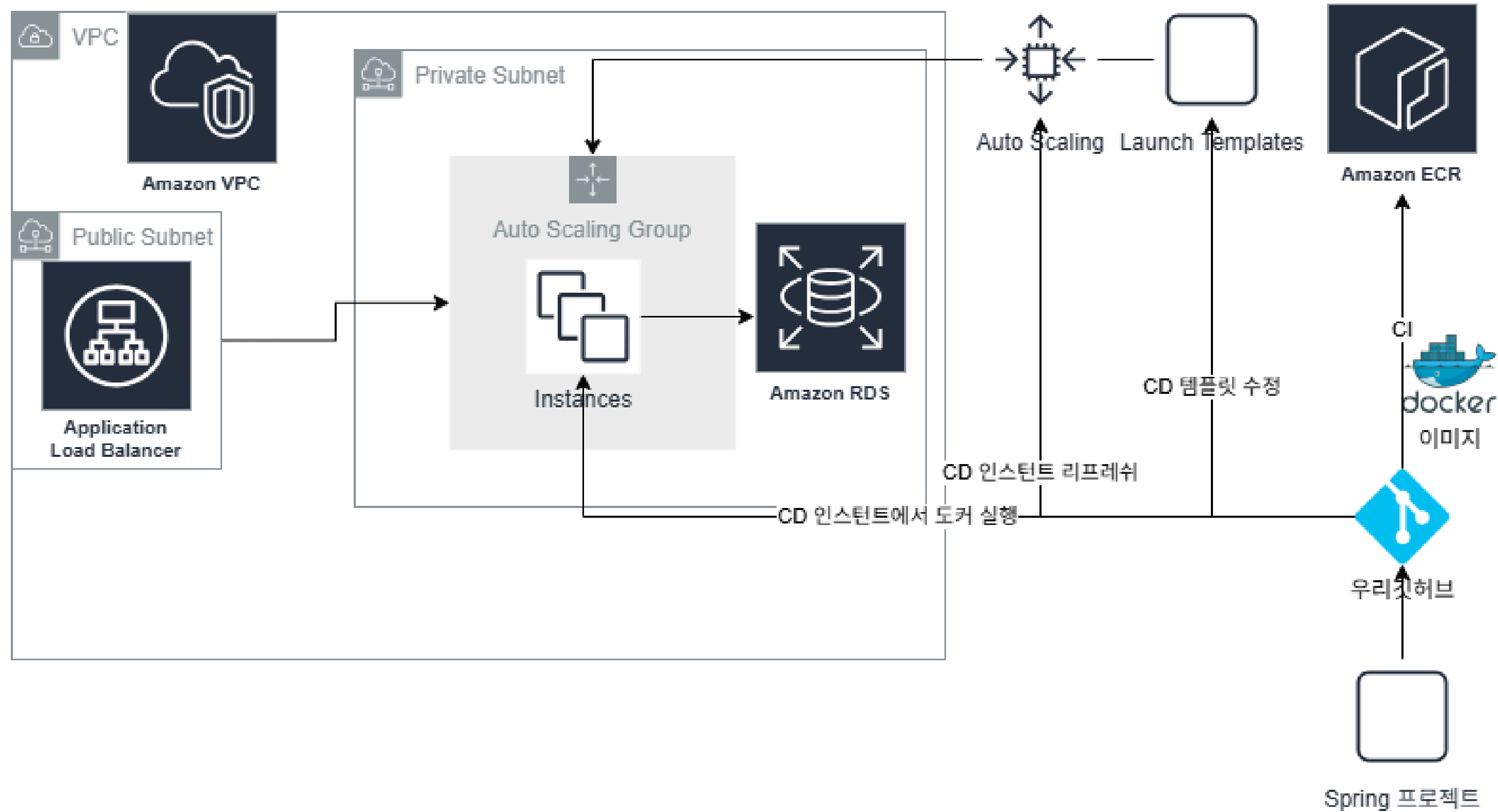
04 시스템 설계 - API



🚦 진행여부	📍 도메인	📄 기능명	📄 API Path	📄 method	📄 담당자	📄 종류
● 완료	구독	📄 플랜 목록 조회	/api/subscriptions/plans	GET	이지민	클라 → 서버
● 완료	구독	📄 빌링키 발행	PortOne.requestIssueBillingKey()		박영수	클라 → 포트원
● 완료	구독	📄 구독 생성	/api/subscriptions	POST	이지민	클라 → 서버
● 완료	구독	📄 구독 조회	/api/subscriptions/{subscriptionId}	GET	이지민	클라 → 서버
● 완료	구독	📄 구독 변경	/api/subscriptions/{subscriptionId}	PATCH	이지민	클라 → 서버
● 완료	구독	📄 구독 결제	/api/subscriptions/{subscriptionId}/billing	POST	박영수	클라 → 서버
● 완료	구독	📄 빌링키 결제	api.portone.io/payments/payments/{paymentId}/billing-key	POST	박영수	서버 → 포트원
● 완료	구독	📄 구독 결제 목록	/api/subscriptions/billing	GET	박영수	클라 → 서버
● 완료	데브옵스	📄 CICD		SYSTEM	성기찬	
● 완료	데브옵스	📄 AWS 인프라구축		SYSTEM	성기찬	

빌링키 기반으로 정기 결제와 구독 상태를 관리하도록 설계

04 인프라 아키텍처(AWS)



주문 - 결제 분리 구조



- 주문과 결제를 분리하여 설계
 - 결제 실패/환불 시에도 주문 데이터 유지
 - 상태 관리 독립 처리
- 결제 실패에도 안정적인 주문 관리 가능

포인트 이력 기반 관리



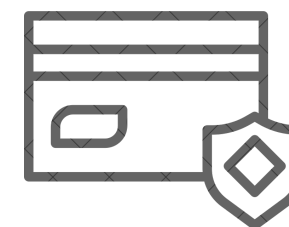
- 잔액이 아닌 거래 이력 기반 관리
 - 적립/차감/사용 내역 추적 가능
 - 데이터 정합성 보장
- 포인트 흐름을 투명하게 관리 가능

구독&빌링키 기반 자동 결제

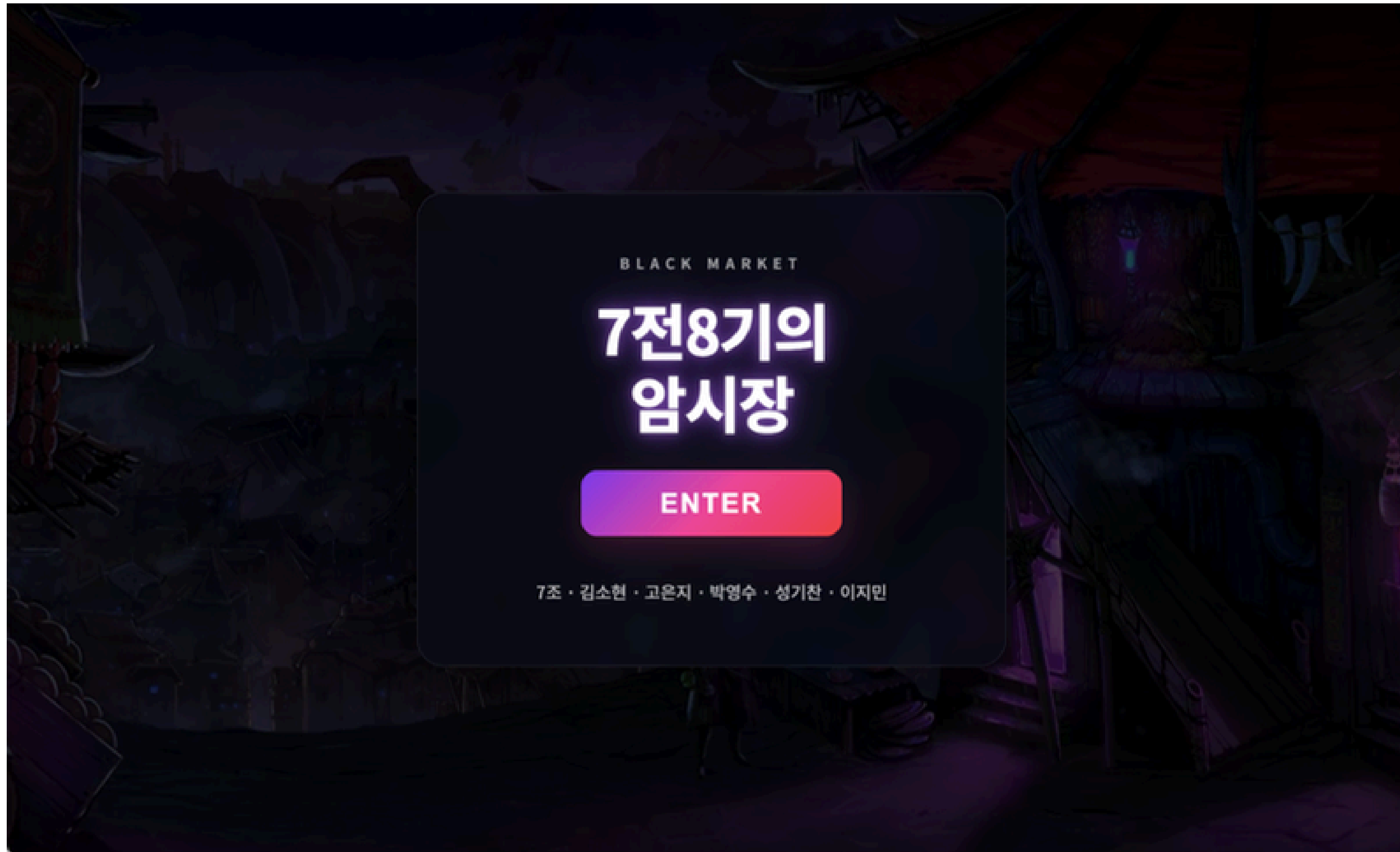


- 빌링키 발급 후 자동 결제 처리
 - 정기 결제 및 구독 관리
 - 플랜 변경 / 해지 지원
- 반복 결제 자동화로 사용자 편의성 향상

결제 검증 & 웹훅 처리



- PortOne 결제 검증 API 활용
 - 위변조 방지
 - webhook 기반 상태 동기화
- 결제 신뢰성 및 시스템 일관성 확보





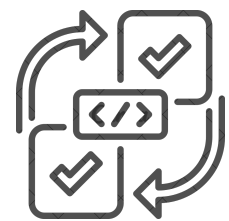
단위 테스트 및 통합 테스트

- src
 - main
 - test/java/com/paymentapp/api
 - auth
 - AuthServiceTest.java
 - payment
 - PaymentServiceTest.java
 - plan
 - PlanServiceTest.java
 - subscription
 - SubscriptionServiceTest.java
 - webhook
 - WebhookServiceTest.java

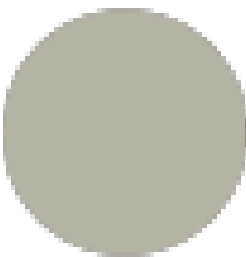
TestCase (통합 테스트)

Payment-App

화면	동작	상태	예상 결과	테스트 결과	확인자	확인 일자
로그인 페이지	유효한 사용자 ID와 비밀번호 입력 후 로그인 버튼 클릭	200	메인 페이지로 이동	Pass	성기찬	2026년 3월 24일
로그인 페이지	유효하지 않은 ID나 비밀번호를 입력 후 로그인 버튼 클릭	400	로그인 실패 메시지 뜨기	Pass	성기찬	2026년 3월 24일
로그인 페이지	구글 로그인 버튼 클릭	200	로그인 성공 후 메인페이지로 이동	Pass	성기찬	2026년 3월 24일
회원가입 페이지	필요한 값들을 입력하고 회원가입 버튼을 클릭	200	회원 가입 후 로그인 페이지로 이동	Pass	성기찬	2026년 3월 24일
회원가입 페이지	이미 가입되어 있는 아이디로 가입시도	400	회원가입 실패 메시지 뜨기	Pass	성기찬	2026년 3월 24일
내 페이지	내 이메일 누르기	200	회원 정보가 조회되기	Pass	성기찬	2026년 3월 25일
상품 페이지	상품 및 재고 선택 후 주문 생성 버튼 클릭	200	주문 페이지로 넘어가기 주문 목록에 새로운 주문 뜨기	Pass	김소현	2026년 3월 24일
상품 페이지	상품별 상세조회 버튼 클릭	200	해당 상품의 상세 정보 뜨기	Pass	김소현	2026년 3월 24일
주문 페이지	주문 목록에서 주문 번호 클릭	200	해당 주문의 주문 상세 정보 뜨기	Pass	김소현	2026년 3월 24일
주문 페이지	내 주문 목록에서 결제 대기 상태인 주문의 결제하기 버튼 클릭	200	결제창 뜨기	Pass	김소현	2026년 3월 24일
주문 페이지	결제창에서 결제 완료하고 결제 버튼 클릭	200	주문 목록에 결제 완료 뜨기	Pass	김소현	2026년 3월 24일
주문 페이지	결제창에서 결제 실패시	500	주문 목록에 취소됨 뜨기	Pass	김소현	2026년 3월 24일
주문 페이지	결제 취소에서 결제 ID 입력 후 결제 취소/환불 버튼 클릭	200	결제 취소 성공! 메시지 뜨기 주문 목록에 환불 완료 뜨기	Pass	김소현	2026년 3월 24일
주문 페이지	결제 취소에서 잘못된 결제 ID 입력 후 결제 취소/환불 버튼 클릭	400	결제 취소 실패 메시지 뜨기	Pass	김소현	2026년 3월 24일
주문 페이지	결제 취소에서 이미 취소된 결제 ID 입력 후 결제 취소/환불 버튼 클릭	500	결제 취소 성공! 메시지 뜨기	Pass	김소현	2026년 3월 24일



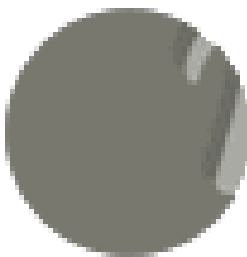
통합 테스트



[1차 테스트]

- 포인트 결제 실패(포인트 미보유)
- 구독 스케줄러 중복 실행 문제
- 마이페이지/주문내역 기능 부족

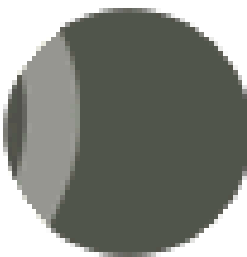
→ 핵심 기능 및 사용자 흐름 검증 필요 확인



[2차 테스트]

- 재구독 실패(customerUid unique 제약 문제)
- 주문 버튼 연속 클릭 시 중복 주문 발생
- ShedLock 테이블 운영 환경 미적용

→ 데이터 제약 조건 및 사용자 입력 제어 필요
→ 운영 환경 설정 문제 확인



[최종 결과]

- 모든 핵심 시나리오 정상 동작
- 결제/구독/주문 흐름 안정화
- 운영 환경까지 고려한 시스템 완성

→ 3차 테스트 PASS



트러블슈팅 #1) 동시성 환경에서 발생한 재고 초과 판매 문제

원인

동시 결제 요청 시
동일 재고에 대한 동시 접근

- 여러 트랜잭션이 같은 재고를 동시에 조회/수정
- 경쟁상태(Race Condition) 발생 → Lost Update 문제

→ 재고 10개인데 15개 이상 판매되는 초과 판매 발생

해결

비관적 락(Pessimistic Lock)
기반 동시성 제어

- SELECT ... FOR UPDATE로 재고 선점
- 트랜잭션 동안 해당 row 접근 차단
- 상품 ID 정렬로 데드락 방지
- 여러 상품을 한 번에 조회(Bulk Lock)로 성능 개선

결과

재고 정합성 확보 및
안정적인 결제 처리

- 초과 판매(Over-selling) 문제 해결
- 동시 요청 상황에서도 일관성 유지
- 결제 실패 시 재고 복구(보상 트랜잭션) 적용

충돌 빈도가 높은 결제 도메인 특성을 고려해 정합성을 우선으로 비관적 락 선택



트러블슈팅 #2) 다중 서버 환경에서 스케줄러 중복 실행 문제

원인

다중 서버 환경에서
@Scheduled가 서버마다
독립적으로 실행

- 동일 시간에 여러 서버가 같은
결제 작업 수행
- 동일 구독 데이터 동시 접근 →
경쟁 상태(Race Condition)
발생

→ 중복 결제 및 데이터 정합성
문제 발생

해결

ShedLock 기반 분산 락 적용

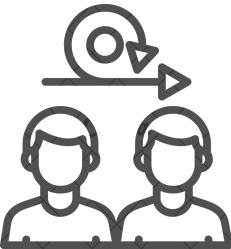
- DB 기반 Lock으로 스케줄러
실행 권한 제어
- 하나의 서버만 Lock 획득 후 작
업 수행
- 다른 서버는 실행 스킵하여 중
복 실행 방지
- 초기 이슈: Lock 테이블 미존재
→ schema 설정으로 해결

결과

스케줄러 단일 실행 보장 및
안정성 확보

- 중복 결제 문제 해결
- 다중 서버 환경에서도 일관성
유지
- 안정적인 정기 결제 처리 가능

DB 기반 락의 한계를 인지하고, 향후 Redis 기반 분산 락으로 개선 예정



회고



김소현

“결제는 실패해도 되지만,
틀리면 안 된다”는 점을
알게 되었고
단순 기능 구현을 넘어서
데이터 정합성과 시스템
안정성을 우선적으로 고
려하는 사고를 갖게 되었
습니다.

철저한 테스트를 통해 프
로젝트의 완성도를 높이
는 경험을 할 수 있어 좋
았습니다.



고은지

기능 하나를
구현하더라도 정합성은
물론, 이력까지 고려해야
되겠다고 느꼈습니다.

질문을 해도 눈치
보이지 않는 분위기와
원활한 소통 덕분에
진행이 빠르고 큰 문제가
없었습니다. 팀원 분들의
도움에 크게 성장하게
되어 감사한 점이
많았습니다.



박영수

단순히 구독 기능을
넘어서 다중환경에서의
동시성 문제와 인프라
자원의 효율성까지
고민해본 뜻 깊은
시간이었습니다.

더불어 팀원들 각자의
장점에 맞춰 역할을
명확히 분배하였고,
활발한 소통을 통해
효율적인 협업 시너지를
낼 수 있었습니다.



성기찬

실무에서도 쉽게 만나기
힘들 만큼 훌륭한
팀원들과 함께 했던 것
같습니다.

2주동안 많은 시간과
정성을 쏟았는데
팀원님들과 튜터님
덕분에 그 시간들이
빛날 수 있었던 것
같습니다.

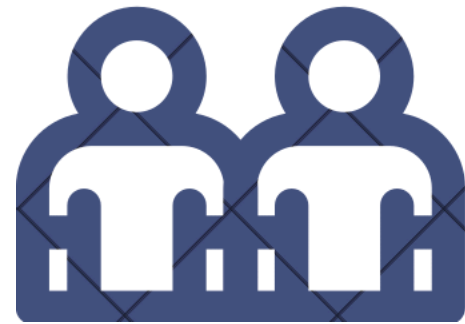


이지민

상품 및 주문과
구독 기능을 담당하며
상태 관리와 흐름 설계의
중요성을 체감했습니다.

기능 구현뿐 아니라
팀원 간 소통과
명확한 역할 분담이
프로젝트 완성도를
높인다는 것을
배웠습니다.

7조 최고



이상으로 발표를 마치겠습니다.
궁금한 점이 있으시면 편하게 질문해주세요



TEAM PROJECT PRESENTATION

감사합니다

김소현

고은지

박영수

성기찬

이지민

